Tweet Sentiment Analysis

Lum Meta - ICT - AI

In this project we aim to make a model that can determine whether a given Tweet has a positive or negative sentiment.

We start by sampling the data set from nltk and then move to cleaning the data by removing punctuation, removing stop words and performing stemming.

```
import nltk
import numpy as np
import random
from nltk.corpus import twitter_samples  # sample Twitter dataset from NLTK
from nltk.tokenize import RegexpTokenizer
from nltk.stem.porter import PorterStemmer
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

Imports

```
tokenizer=RegexpTokenizer(r'\w+')
en_stopwords=set(stopwords.words('english'))
ps=PorterStemmer()

def removeUsername(text):
    return ' '.join(np.array(text.split(' ')[1:]))

def getCleanedText(text):
  texttext=text.lower()
  #tokenize
  tokens=tokenizer.tokenize(text)
  new_tokens=[token for token in tokens if token not in en_stopwords]
  stemmed_tokens=[ps.stem(tokens) for tokens in new_tokens]
  clean_text=" ".join(stemmed_tokens)
  return clean_text
```

Setting up a cleaning function and a function to remove the username for the data

```
Positive_tweets = [removeUsername(p) for p in all_positive_tweets]
Negative_tweets = [removeUsername(n) for n in all_negative_tweets]

Positive_tweets = [getCleanedText(p) for p in Positive_tweets]
Negative_tweets = [getCleanedText(n) for n in Negative_tweets]
```

Using the cleaning functions on the data

Before cleaning - @TomParker oh wow!! That is beautiful tom :)

After cleaning - oh wow that beauti tom

Next, we need to split the data into training and testing sets

```
Positive_tweets_train = np.array(Positive_tweets[:4000])
Positive_tweets_test = np.array(Positive_tweets[1000:])

Negative_tweets_train = np.array(Negative_tweets[:4000])
Negative_tweets_test = np.array(Negative_tweets[1000:])

Training_set = np.concatenate((Positive_tweets_train, Negative_tweets_train))
Testing_set = np.concatenate((Positive_tweets_test, Negative_tweets_test))
```

Splitting the datasets and combining the positive and negative sets together

Now we need to generate labels for the data before shuffle the lists

```
#Generate a list of labels for both sets
Training_labels = np.concatenate((np.ones(len(Positive_tweets_train), dtype=int),
                                  np.zeros(len(Negative_tweets_train), dtype=int)))
Testing_labels = np.concatenate((np.ones(len(Positive_tweets_test), dtype=int),
                                 np.zeros(len(Negative_tweets_test), dtype=int)))

#Asigning the label and shuffling the list
Training_combined = list(zip(Training_set, Training_labels))
random.shuffle(Training_combined)
Training_set[:], Training_labels[:] = zip(*Training_combined) #splitting the tweets from the labels

Testing_combined = list(zip(Testing_set, Testing_labels))
random.shuffle(Testing_combined)
Testing_set[:], Testing_labels[:] = zip(*Testing_combined) #splitting the tweets from the labels
```

This is done because the nltk provides the data in batches of positive and negative tweets, training the model with only positive tweets and then only negative tweets won't lead to good results that's why we shuffle the order of the tweets after labeling so when the data is split up again the position of the

tweet and the label will correspond to each other in their separate lists.

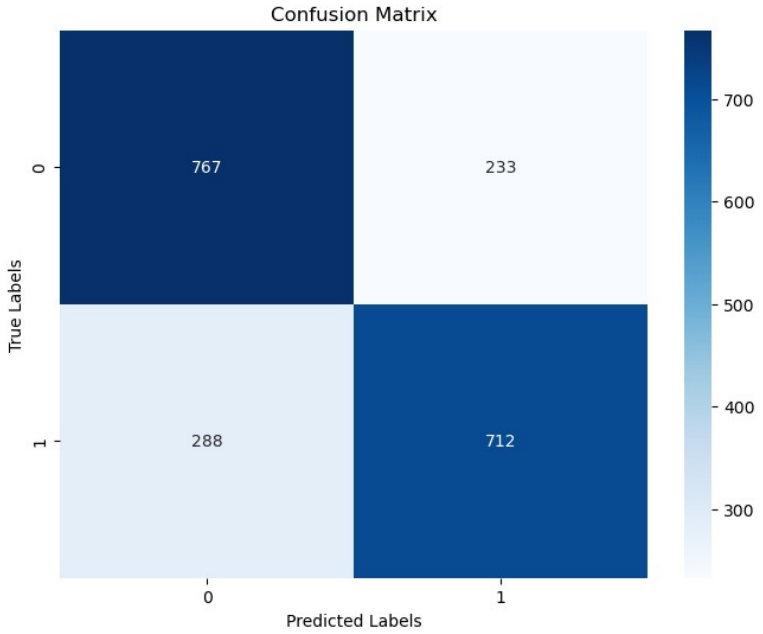
Now we need to convert the data into numerical values and we do that with CountVectorizer and ngram_range to extract both individual words and word pairs as features

```
cv=CountVectorizer(ngram_range=(1,2))
Training_vect=cv.fit_transform(Training_set).toarray()
Testing_vect=cv.transform(Testing_set).toarray()
```

Now that the data is set up, we can finally fit and test the model

```
mlb=MultinomialNB()
mlb.fit(Training_vect,Training_labels)
Predictions=mlb.predict(Testing_vect)
```

For the firs model we are fitting a Multinomial Naive Bayes classifier model and with this we got an Accuracy of 73.95%



With this confusion matrix we can see that the model is slightly better at correctly identifying positive tweets compared to negative tweets

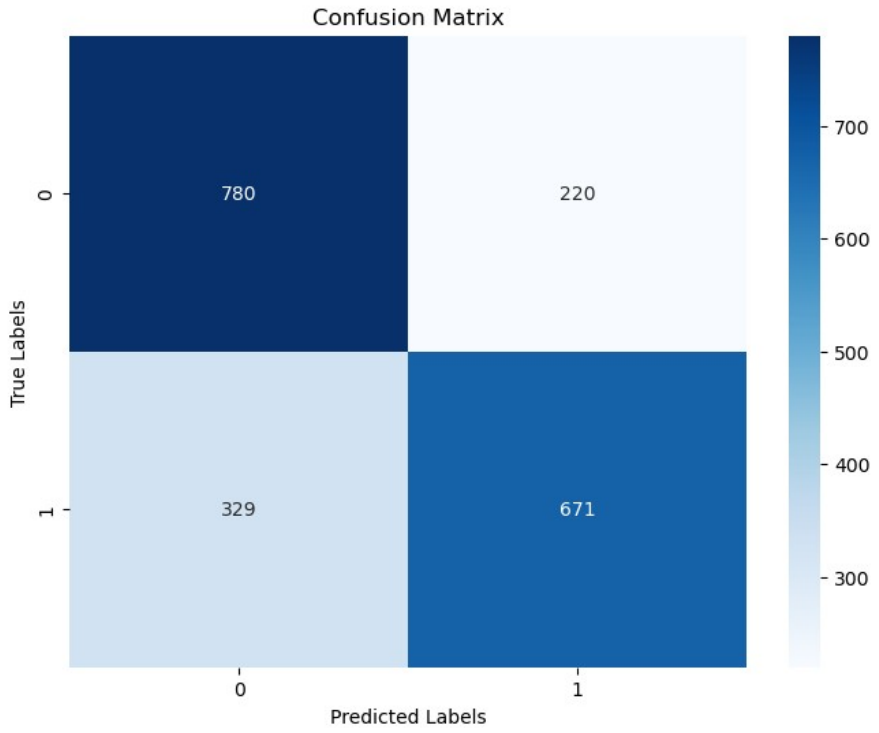
Next I will try training a Logistic Regression model using the *liblinear* solver, which is well suited for sentiment analysis

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(solver='liblinear')
lr.fit(Training_vect,Training_labels) # fit the model
preds = lr.predict(Testing_vect) # make predictions
```

With this model we got this result: Accuracy:

72.55%



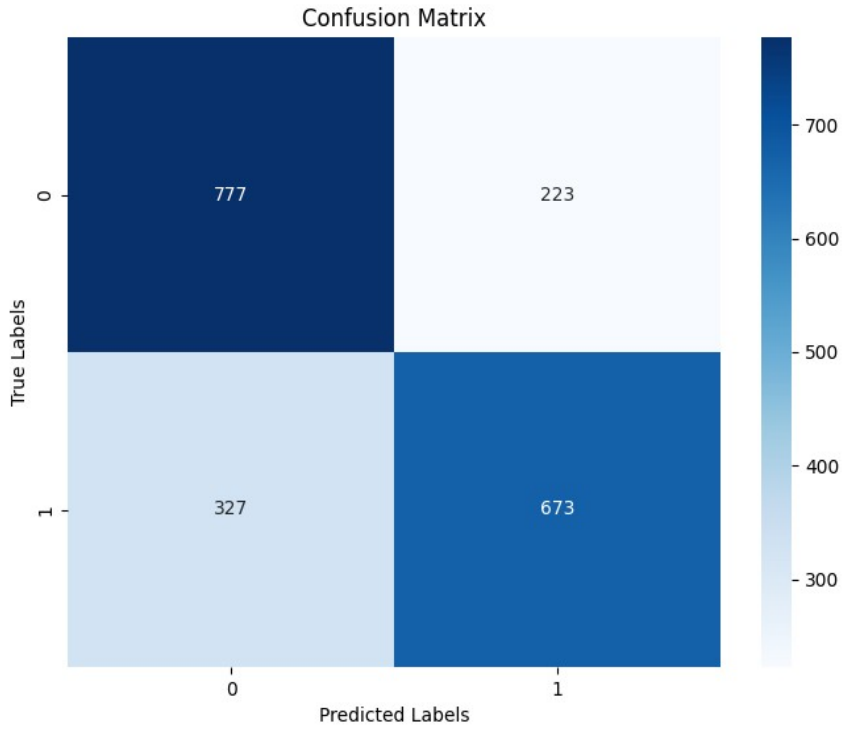
So, this time we can see that the model accuracy has dropped by little bit and the model is again better at classifying the positive tweets and was able to classify more than the last model, but the accuracy of negative tweets dropped a lot

I tried other Logistic Regression models using different solvers and penalty rules, but they all lead to Similar results

Here I used the sega solver with l1 penalty rule

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(solver='saga', penalty='l1', max_iter=1000)
lr.fit(Training_vect,Training_labels) # fit the model
preds = lr.predict(Testing_vect) # make predictions
```

and got these results:



Accuracy: 72.5%
As we can see the overall accuracy is the same, with the difference being the model got a few more negative tweets right and a few positive tweets wrong.

After that I tried an SVM model

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

svm_model = SVC(kernel='linear')
svm_model.fit(Training_vect, Training_labels)
svm_predictions = svm_model.predict(Testing_vect)
```

which also provided similar results with an accuracy of 71.45%

Now instead of using raw word counts with CountVectorizer, we try TF-IDF, which not only considers how often a word appears in a tweet, but also how unique that word is across all tweets.

```
from sklearn.feature_extraction.text import TfidfVectorizer
cv = TfidfVectorizer(ngram_range=(1,2))
```

Doing this the results are similar again, I'll put the results on a table:

Model	Result
MultinomialNB()	73.7%
LogisticRegression(solver='liblinear')	73.05%
lr = LogisticRegression(solver='saga', penalty='l1', max_iter=1000)	71.15%
SVC(kernel='linear')	73.3%

Next I started to experiment with some simple neural networks

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import accuracy_score

model = Sequential()
model.add(Dense(16, activation='relu', input_shape=(Training_vect.shape[1],)))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer=Adam(learning_rate=0.001), loss='binary_crossentropy', metrics=['accuracy'])

model.fit(Training_vect, Training_labels, epochs=10, batch_size=32, validation_split=0.2)
y_pred = (model.predict(Testing_vect) > 0.5).astype("int32")
```

This gave a 70.3% accuracy, but looking at the rest of the data that is printed when training

Epoch 1/10 - accuracy: 0.6148 - loss: 0.6825 - val_accuracy: 0.7188 - val_loss: 0.6037
Epoch 2/10 - accuracy: 0.8925 - loss: 0.4463 - val_accuracy: 0.7381 - val_loss: 0.5131
Epoch 3/10 - accuracy: 0.9585 - loss: 0.1722 - val_accuracy: 0.7356 - val_loss: 0.5201
Epoch 4/10 - accuracy: 0.9787 - loss: 0.0783 - val_accuracy: 0.7281 - val_loss: 0.5466
Epoch 5/10 - accuracy: 0.9836 - loss: 0.0496 - val_accuracy: 0.7281 - val_loss: 0.5660
Epoch 6/10 - accuracy: 0.9841 - loss: 0.0386 - val_accuracy: 0.7219 - val_loss: 0.5854
Epoch 7/10 - accuracy: 0.9826 - loss: 0.0381 - val_accuracy: 0.7275 - val_loss: 0.6192
Epoch 8/10 - accuracy: 0.9860 - loss: 0.0314 - val_accuracy: 0.7300 - val_loss: 0.6469

Epoch 9/10 - accuracy: 0.9885 - loss: 0.0259 - val_accuracy: 0.7275 - val_loss: 0.6653
Epoch 10/10 - accuracy: 0.9843 - loss: 0.0292 - val_accuracy: 0.7256 - val_loss: 0.6638

The accuracy being so high and the val_accuracy having such a big drop shows clear signs of overfitting

Also in the dataset there's a lot of very short tweets which don't provide any context so it's easy for those to cause problems when training, i will remove all tweets that are less then 7 words long, this leaves us with 3770 samples in positive tweets and 3526 in negatives

```
all_positive_tweets = [tweet for tweet in all_positive_tweets if len(tweet.split()) >= 7]
all_negative_tweets = [tweet for tweet in all_positive_tweets if len(tweet.split()) >= 7]
```

This also didn't lead to any improvements, i also tried to remove all the tweets with less the 10 words and got similar results

To try and work against overfitting I added dropout layers, after trying different combinations of layer sizes, number of epochs and batches this combination gave the best results(before removing the short tweets)

```
model = Sequential()
model.add(Dense(8, activation='relu', input_shape=(Training_vect.shape[1],)))
model.add(Dropout(0.5))

model.add(Dense(4, activation='relu'))
model.add(Dropout(0.5))

model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer=Adam(learning_rate=0.001), loss='binary_crossentropy', metrics=['accuracy'])
model.fit(Training_vect, Training_labels, epochs=3, batch_size=8, validation_split=0.1)
```

This gave an accuracy of 73.6%
The model is still overfitting and hard to improve

Since the data is vectorized we can use a random forest classifier

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier()
rf_model.fit(Training_vect, Training_labels)

rf_pred = rf_model.predict(Testing_vect)
```

This model had an accuracy of 70.45% (Before removing short tweets)

And after removing the short tweets the results got worse with 69.6% accuracy

At the end the best performing model was the multinomial Naive Bayes classifier, with 73.95% accuracy, I think this could be better improved with a larger dataset and maybe removing some samples that were neutral.